DATAVERSE AI

Complete Technical Implementation Report

Models, datasets, libraries, architecture, APIs, Supabase, security, testing, and deployment

Repository	C:\Users\mouav\OneDrive\Desktop\FINAL3
Audit date	13 July 2026
Audience	Technical reviewers, project supervisors, developers, and defense panel
Evidence basis	Source code, dependency manifests, model metadata, serialized pipelines, dataset profiling, tests, lint, and production build
Document status	Repository-audited implementation report

This report describes what is actually present in the repository. Optional or configured-but-unused components are explicitly labeled.

Document control

Table 1: Document control and interpretation rules

Item	Value
Scope	The current repository and its declared/local runtime dependencies
Primary branch state	Working tree as inspected on 13 July 2026
Active	Imported or reached by the current Next.js to FastAPI session flow
Conditional	Runs only when a feature is requested or required configuration exists
Optional	Present in code/configuration but not required for deterministic analysis
Legacy/compatibility	Still mounted or tested, but not the main frontend path
Not established	No repository evidence proves external production configuration or live service state

Table of contents

1. Technical summary

DataVerse AI is a full-stack, deterministic-first dataset analysis system. A Next.js 15 frontend communicates with a FastAPI backend. The backend accepts CSV and Excel files, profiles and semantically maps their columns, computes business metrics with Pandas and NumPy, trains scikit-learn models when requested and safe, explains eligible models with SHAP or feature importance, generates HTML/PDF reports, and persists identity/data through Supabase or a local fallback.

Table 2: Most important implementation findings

Finding	Confirmed implementation	Implication
Deterministic calculations	Pandas, NumPy, and scikit-learn compute numbers. LLMs only plan/refine language.	The numerical path works without any LLM key.
Two model paths	Saved retail artifacts exist, while the live application trains models per uploaded dataset.	Saved .pkl files are evidence artifacts, not the current inference endpoint.
Saved model quality	Ridge is saved for total_sales and profit; RandomForestClassifier is saved for stockout_flag.	Metrics must be interpreted with leakage and class-imbalance caveats.
Real authentication	Supabase Auth handles password storage, confirmation links, login, refresh, and user lookup.	Application code never stores plaintext passwords.
Persistence	Supabase stores metadata and private files when configured; local JSON/filesystem fallback otherwise.	A working local app does not prove Supabase is configured.
Verified features	Receipts, SHA-256 certificates, quality doctor, what-if, root-cause, XAI, and reports are implemented.	Outputs can include traceable formulas and reproducibility checks.
Current verification	214 backend tests, frontend lint, and the production Next.js build pass.	The inspected codebase is internally consistent at the tested level.

Critical interpretation

The saved total_sales model reports R2 = 1.0 because engineered inputs price_qty and discount_value almost exactly reconstruct total_sales. This is target leakage for forecasting future sales and should not be presented as real predictive performance.

Evidence: models/model_metadata.json; scripts/train_retail_mart.py; dataverse_backend/app/services/modeling.py; test/build outputs from this audit

2. Scope, audit method, and evidence

The report answers: which components, datasets, algorithms, models, libraries, services, security controls, routes, and deployment mechanisms are present, and which of them participate in the current application flow?

- Inspected source code under frontend/, dataverse_backend/app/, scripts/, models/, supabase/, and tests/.
- Read package.json, package-lock.json, all backend requirement manifests, Dockerfiles, environment examples, and SQL migrations.
- Profiled data/retail_mart_processed_v1.csv directly with Pandas.
- Loaded the three local serialized scikit-learn pipelines to confirm estimator types and pipeline steps.
- Compared saved metrics and feature importance with the training script and calculated leakage checks.
- Enumerated FastAPI decorators and frontend API calls.

- Ran the full backend test suite, frontend lint, and Next.js production build.

Evidence boundary

No secret values were printed or embedded. The audit does not prove the state of the external Supabase dashboard, email provider, DNS, hosted deployment, or production environment variables.

3. System architecture and runtime flow

The active architecture has six essential parts: the user, Next.js frontend, FastAPI API, two analysis agents, deterministic analytics engine, and Supabase. Results return through the same authenticated API path used for requests.

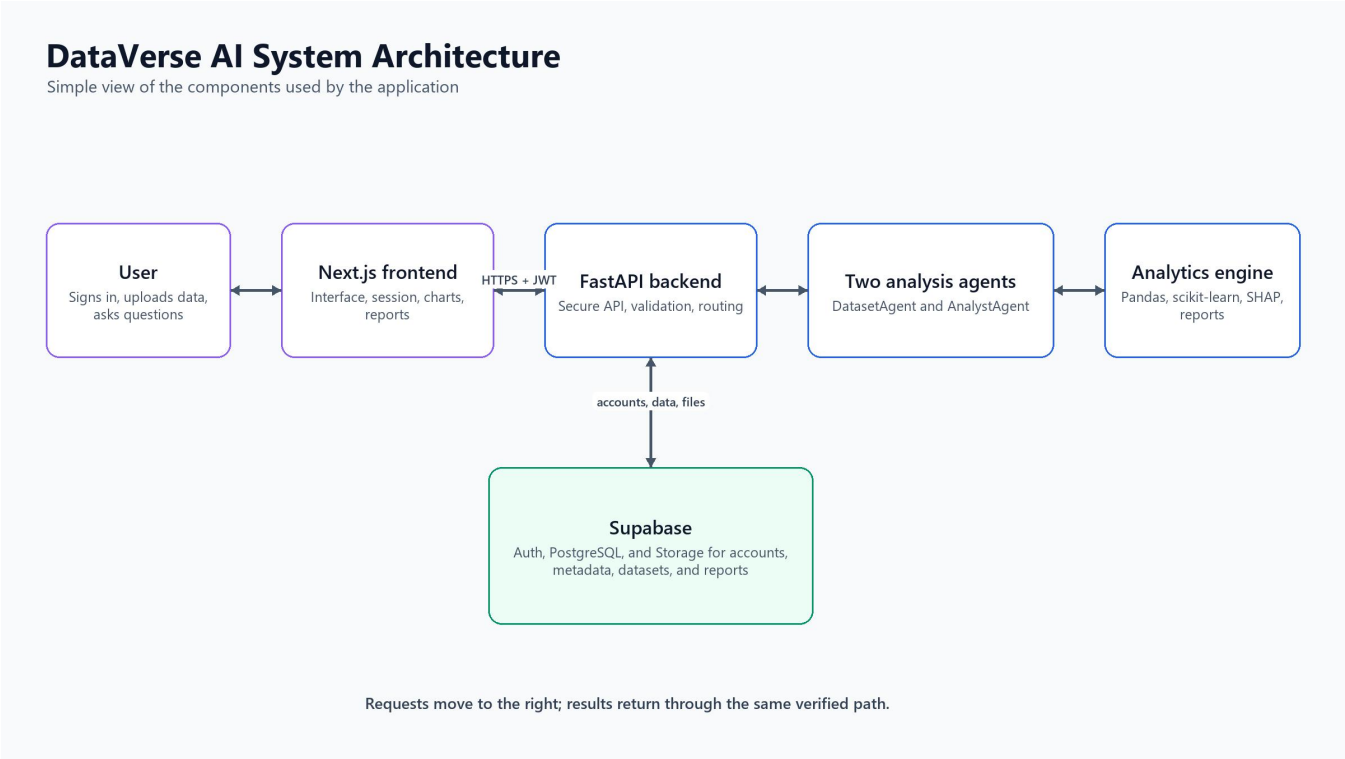

Figure 1: DataVerse AI system architecture based on used components

Evidence: output/architecture/dataverse-production-architecture.png; frontend/lib/dataverse-api.ts; dataverse_backend/app/main.py; dataverse_backend/app/services/session_service.py

3.1 End-to-end user flow

Table 3: Runtime processing stages

Stage	Component	Processing performed	Main output
1	Supabase Auth via FastAPI	Signup validates password, sends confirmation link, logs in confirmed users, refreshes sessions.	JWT access token, refresh token, public user profile
2	Next.js dashboard	Creates a session and uploads CSV/XLS/XLSX over multipart HTTP.	Session ID and dataset upload request
3	FastAPI + SessionService	Validates size/extension, checks ownership, parses data, stores file and metadata.	Dataset ID, profile, semantic map

Stage	Component	Processing performed	Main output
4	DatasetAgent	Normalizes headers, profiles shape/types/missingness, and computes initial quality evidence.	Profile and data-quality payload
5	AnalystAgent + AnalysisPipeline	Plans intent, computes metrics/EDA/trends/correlations/outliers, and conditionally trains a model/XAI.	Facts, tables, charts, receipts, model evidence
6	ReportNarrator/Generator	Organizes deterministic facts and optionally improves wording with an LLM.	Chat answer, HTML report, PDF report
7	Supabase/local persistence	Stores sessions/messages/metadata and private dataset/report files.	Durable metadata and signed/local report access
8	Frontend evidence panels	Displays KPIs, charts, quality issues, agent trace, root cause, what-if, certificates, and reports.	Interactive analysis workspace

3.2 Active and compatibility API paths

The main frontend uses /api/sessions/* and /api/datasets/* routes. Legacy /api/upload and /api/analyze/* routes remain mounted and are covered by tests. This increases compatibility but also increases the maintenance and security-review surface.

4. Repository and technology stack

Table 4: Repository areas

Area	Purpose	Status
frontend/	Next.js App Router pages, dashboard components, API client, auth context, local HTTPS launcher	Active
dataverse_backend/app/api/	FastAPI route definitions and request schemas	Active plus compatibility routes
dataverse_backend/app/agents/	DatasetAgent and AnalystAgent wrappers; older agents also remain	Mixed active/legacy
dataverse_backend/app/services/	Core analytics, orchestration, persistence, security, reporting, and AI logic	Primary implementation
dataverse_backend/tests/	Backend unit, integration, security, API, reporting, and agentic tests	214 passing
supabase/migrations/	Chat/session database and storage schema	Deployment input
data/	Processed 35,000-row retail dataset	Training/evaluation input
models/	Three saved sklearn pipelines and metadata	Offline artifacts; not loaded by live app
scripts/	Launch, smoke test, model training, academic documents, and this technical report	Tooling
docs/	Architecture, setup, evaluation, deployment, and historical completion documents	Documentation; some statements may be stale

4.1 Primary stack

Table 5: Primary application technologies

Layer	Technology	Installed/deployed role
Frontend	Next.js 15.5.20, React 19.2.7, TypeScript 5.9.3	App Router pages and client dashboard
Styling	Tailwind CSS 4.1.11, clsx, tailwind-merge, Motion, Lucide	Responsive UI, animation, icons
Backend	Python 3.12, FastAPI 0.135.2, Uvicorn 0.42.0	REST API and ASGI server
Data	Pandas 2.3.3, NumPy 2.3.5, OpenPyXL 3.1.5	Dataframes, numerical processing, Excel input
Machine learning	scikit-learn 1.8.0, SHAP 0.51.0	Preprocessing, candidates, metrics, explanations
Reports	Jinja2 3.1.6, ReportLab 4.5.1	HTML/PDF output
Identity/data platform	Supabase Auth, PostgreSQL REST, Storage	Accounts, metadata, private file objects
External text AI	OpenAI, Gemini, Anthropic, DeepAnalyze, DeepSeek configuration	Optional planning and narration only
Verification	pytest, ESLint, Next build	Backend and frontend quality gates

5. Frontend implementation

The frontend is a Next.js App Router application. Public marketing/authentication pages are statically generated. The dashboard is a client-heavy workspace that orchestrates sessions, uploads, chat, evidence panels, and report links through a typed API client.

Table 6: Frontend pages and important components

File	Surface	Responsibility
app/page.tsx	/	Marketing landing page
app/about/page.tsx	/about	Project principles, pipeline, and technology overview
app/features/page.tsx	/features	Feature catalog
app/signup/page.tsx	/signup	Validated signup and email-confirmation waiting state
app/login/page.tsx	/login	Login and confirmed-email feedback
app/dashboard/page.tsx	/dashboard	Protected shell that renders the analysis workspace
components/dashboard/DashboardApp.tsx	Dashboard	Upload, session, chat, analysis, reports, XAI, and view-state orchestration
components/DropZone.tsx	Upload	Drag/drop and extension checks for CSV/XLS/XLSX
components/ThinkingTrace.tsx	Progress	Renders live Server-Sent Events pipeline steps
components/dashboard/KpiCard.tsx	Evidence	KPI value plus provenance drill-down
components/dashboard/VerificationPanel.tsx	Evidence	Audit receipt inspection
components/dashboard/CertificateCard.tsx	Evidence	Reproducibility certificate validation
components/dashboard/QualityDoctorPanel.tsx	Quality	Issue selection and deterministic cleaning
components/dashboard/WhatIfPanel.tsx	Analysis	Interactive percentage scenario inputs
components/dashboard/RootCausePanel.tsx	Analysis	Driver decomposition and price/volume/mix display
components/dashboard/AgentTracePanel.tsx	Agent	Plan-act-observe tool trace

5.1 Authentication state

- AuthProvider calls only the FastAPI authentication routes; the browser does not call Supabase directly.
- The access token, refresh token, and display session are kept in browser localStorage under dataverse.token, dataverse.refreshToken, and dataverse.session.
- On reload, /api/auth/me validates the access token. A failed access token triggers /api/auth/refresh when a refresh token exists.
- The dashboard route redirects unauthenticated users to /login at the client layer.
- API requests send Authorization: Bearer <token>. A legacy X-Dataverse-User workspace ID is also generated and sent.

5.2 Local HTTPS

npm run dev starts FastAPI on 127.0.0.1:8000 and Next.js with --experimental-https on 127.0.0.1:3000. Browser API traffic uses the secure same-origin /backend/api proxy, while Next.js communicates server-side with the local HTTP backend.

Evidence: frontend/scripts/dev.mjs; frontend/next.config.ts; frontend/lib/apiConfig.ts; frontend/lib/auth.tsx; frontend/lib/dataverse-api.ts

6. Backend API and orchestration

FastAPI mounts authentication, core, session, dataset, report, storage, and legacy analysis routers. SessionService is the central application service: it enforces session ownership, loads datasets, persists messages and agent runs, invokes AnalysisPipeline, and generates reports.

Table 7: Backend boundary controls

Control	Implementation
CORS	Configured allow-list with optional deployment regex; credentials allowed
Compression	GZip for responses of 500 bytes or more
Rate limiting	POST auth/upload/message routes use an in-memory sliding window
Security headers	nosniff, frame denial, strict-origin referrer policy, disabled camera/microphone/geolocation
Ownership	SessionService.ensure_access blocks user ID mismatches
Errors	Development returns diagnostic details; production returns a generic unexpected-error message
Progress	Per-session Server-Sent Events stream reports pipeline stages

6.1 Complete API catalog

Table 8: FastAPI endpoint catalog

Method	Path	Purpose
GET	/health/live	Public liveness check
GET	/api/health	API health and configuration status
POST	/api/auth/signup	Create a Supabase password user and require email confirmation
POST	/api/auth/login	Exchange email/password for Supabase access and refresh tokens
POST	/api/auth/resend-signup	Resend the signup confirmation link
POST	/api/auth/refresh	Refresh an expired access session
GET	/api/auth/me	Validate the bearer token and return the public user profile
POST	/api/sessions	Create a chat/analysis session
GET	/api/sessions	List recent sessions for the resolved identity
GET	/api/sessions/{session_id}	Load a session with messages, datasets, agent runs, and reports
PATCH	/api/sessions/{session_id}	Update session metadata such as title

Method	Path	Purpose
DELETE	/api/sessions/{session_id}	Delete an authorized session
POST	/api/sessions/{session_id}/datasets/upload	Upload CSV/XLS/XLSX into a session
GET	/api/sessions/{session_id}/datasets	List datasets belonging to a session
POST	/api/sessions/{session_id}/analyze	Run the main analysis pipeline
POST	/api/sessions/{session_id}/datasets/{dataset_id}/clean	Apply selected Data Quality Doctor fixes and re-analyze
POST	/api/sessions/{session_id}/datasets/{dataset_id}/verify	Recompute and verify a reproducibility certificate
POST	/api/sessions/{session_id}/datasets/{dataset_id}/whatif	Run a receipt-backed percentage scenario
POST	/api/sessions/{session_id}/datasets/{dataset_id}/investigate	Run deterministic root-cause analysis
POST	/api/sessions/{session_id}/messages	Ask a follow-up question against a dataset
GET	/api/sessions/{session_id}/progress/stream	Stream analysis progress over Server-Sent Events
POST	/api/sessions/{session_id}/reports/{report_id}/renarrate	Re-run narration without recomputing analytical facts
GET	/api/sessions/{session_id}/agent-runs	List persisted agent executions
GET	/api/sessions/{session_id}/reports	List reports for a session
POST	/api/sessions/{session_id}/reports/generate	Generate HTML/PDF analysis outputs
GET	/api/reports/{report_id}/download	Return a local file or redirect to a signed Supabase URL
GET	/api/datasets	Compatibility list of recent datasets
POST	/api/datasets/upload	Compatibility upload that creates a new session
GET	/api/datasets/{dataset_id}	Compatibility dataset metadata lookup
GET	/api/datasets/{dataset_id}/profile	Return a stored dataset profile
POST	/api/datasets/{dataset_id}/ask	Compatibility direct question endpoint
DELETE	/api/datasets/{dataset_id}	Compatibility dataset deletion
GET	/api/storage/status	Report Supabase or local persistence mode
POST	/api/upload	Legacy upload/profile endpoint
GET	/api/session/{session_id}	Legacy session lookup
DELETE	/api/session/{session_id}	Legacy session deletion
POST	/api/analyze/upload	Legacy immediate full analysis
POST	/api/analyze/query	Legacy query against an uploaded session

Evidence: dataverse_backend/app/main.py; dataverse_backend/app/api/.py*

7. Agents and analytical pipeline

7.1 DatasetAgent

DatasetAgent owns ingestion. It rejects empty files, files larger than MAX_UPLOAD_SIZE_MB (50 MB by default), and extensions other than .csv, .xlsx, or .xls. It parses the file, normalizes blank/duplicate headers, stores the dataframe, and returns profile and quality evidence.

7.2 AnalystAgent

AnalystAgent wraps AnalysisPipeline. It receives the dataframe, semantic map, query, prediction target/task preferences, XAI flag, and report flag. It returns deterministic analysis facts plus optional narrative/report outputs.

7.3 AnalysisPipeline stage inventory

Table 9: AnalysisPipeline methods and algorithms

Stage	Primary module	Method
Coercion/profile	data_profiler.py	Type detection, missing masks, preview, distributions, cardinality, semantic roles
Dataset type	semantic_mapper.py + dataset_classifier.py	Rule-based role and sample inspection; optional LLM refinement
Planning	query_planner.py + analysis_planner.py	Intent, metric, dimension, filters, report mode, prediction need
Business metrics	business_metrics.py	SUM, derived profit/margin/AOV, date/dimension groupings, rankings
Quality	data_quality.py + quality_doctor.py	Missingness, duplicates, constants, high cardinality, type issues, fixes
EDA	data_quality.py	describe(), frequency tables, histograms, IQR outliers
Relationships	data_quality.py	Pearson correlation matrix for numeric columns
Trends	data_quality.py	Date aggregation and NumPy first-degree polyfit slope
Prediction	target_inference.py + modeling.py	Safe target ranking, preprocessing, holdout comparison of candidate estimators
Explainability	xai.py + counterfactual.py	Tree SHAP or importance fallback, plus single-feature counterfactual search
Verification	provenance.py + audit.py + certificate.py	Formula receipts and SHA-256 fingerprints
Advanced analysis	root_cause.py + whatif.py	Period driver decomposition and deterministic scenario recomputation
Reporting	report_composer.py + report_generator.py	De-duplicated HTML/PDF report with validated charts/tables

7.4 Dataset types recognized

Semantic classification supports mart_sales, retail_sales, invoice_sales, ecommerce_orders, pos_transactions, transaction_ledger, inventory, food_dataset, customer_sales, and generic_tabular. RETAIL_ONLY_UPLOADS defaults to true, so production-like configuration can reject unsupported domains even though generic analysis code exists.

Evidence: dataverse_backend/app/agents/dataset_agent.py; dataverse_backend/app/agents/analyst_agent.py; dataverse_backend/app/services/analysis_pipeline.py; dataverse_backend/app/services/semantic_mapper.py

8. Dataset inputs and processing

8.1 User-uploaded datasets

Table 10: Supported upload behavior

Property	Implementation
Formats	CSV, XLSX, XLS
Default maximum	50 MB
CSV parsing	Delimiter sniffing, sectioned-table detection, repair fallback for inconsistent rows
Excel parsing	Pandas read_excel through the installed Excel engine
Header handling	Whitespace cleanup and generated names for blank columns
Persistence	Per-dataset pickle and CSV, metadata JSON, semantic-map JSON; Supabase private object copy when configured
Validation	Non-empty dataframe, supported extension, domain guard when enabled
Model minimum	At least 30 rows for runtime prediction by default

8.2 Included retail training dataset

Table 11: retail_mart_processed_v1.csv profile

Measure	Observed value
Rows	35,000
Columns	21
File size	3,113,988 bytes (about 2.97 MiB)
In-memory dataframe size	5,880,132 bytes (about 5.61 MiB)
Missing cells	0
Duplicate rows	0
total_sales	Mean 64.2475; median 46.32; min 0; max 495.44; 14,187 unique values
profit	Mean 9.6268; median 6.34; min 0; max 101.63; 4,186 unique values
stockout_flag	7,053 positive (20.15%) and 27,947 negative (79.85%)
Data encoding	All 21 columns are numeric; category label mappings are not included

Table 12: Included retail dataset column dictionary

Column	dtype	Meaning and caveat
store_id	int64	Store identifier; excluded from saved models because names ending in _id are treated as

Column	dtype	Meaning and caveat
		identifiers.
region	int64	Encoded region category. The repository does not include a code-to-label mapping.
city	int64	Encoded city category; seven observed codes.
category	int64	Encoded product category; four observed codes.
subcategory	int64	Encoded product subcategory; ten observed codes.
unit_price	float64	Unit selling price.
quantity	int64	Units in the transaction.
discount	float64	Discount fraction applied to the transaction.
total_sales	float64	Post-discount sales value. Approximately price_qty - discount_value, rounded to cents.
profit	float64	Transaction profit target used by one saved regression artifact.
customer_type	int64	Encoded customer segment.
payment_method	int64	Encoded payment method.
online_order	int64	Binary indicator for online order status.
stockout_flag	int64	Binary stockout target; 7,053 positive and 27,947 negative rows.
weekday	int64	Encoded weekday.
month	int64	Month number.
year	int64	Calendar year.
hour	int64	Hour of day.
price_qty	float64	Engineered value equal to unit_price multiplied by quantity.
discount_value	float64	Engineered value equal to price_qty multiplied by discount.
profit_margin	float64	Engineered profit ratio.

Dataset limitation

The repository contains only the processed numeric dataset. Without the raw source and encoding dictionary, region, city, category, subcategory, customer_type, and payment_method codes cannot be translated back to business labels.

Evidence: data/retail_mart_processed_v1.csv; scripts/train_retail_mart.py

9. Machine-learning models

9.1 Live per-upload modeling

The live application does not use a fixed prediction model. When prediction is explicitly requested, implied by the query, or enabled with a target confidence at or above 0.65, modeling.py builds a model from the uploaded dataset.

Table 13: Live model training design

Aspect	Regression	Classification
Target detection	Numeric, sufficiently high-cardinality target	Boolean/low-cardinality numeric or categorical target
Dummy baseline	DummyRegressor(strategy=mean)	DummyClassifier(strategy=most_frequent)
Candidate 1	Ridge(random_state=42)	LogisticRegression(max_iter=1000, class_weight=balanced)
Candidate 2	RandomForestRegressor(80 trees, max_depth=8, random_state=42)	RandomForestClassifier(80 trees, max_depth=8, class_weight=balanced, random_state=42)
Split	75% train / 25% test, random_state=42	75% train / 25% test, stratified when each class has at least two rows
Selection	Highest test R2 among non-dummy candidates	Highest weighted test F1 among non-dummy candidates
Metrics	R2, RMSE, MAE	Accuracy, weighted F1, confusion matrix
Numeric preprocessing	Median imputation + StandardScaler	Median imputation + StandardScaler
Categorical preprocessing	Most-frequent imputation + one-hot encoding	Most-frequent imputation + one-hot encoding
Safety gates	Minimum 30 rows, identifiers/constants/high-cardinality and name-overlap leakage excluded, max 200 estimated one-hot features	Same gates plus imbalance warning when one class is at least 90%

9.2 Saved retail model artifacts

scripts/train_retail_mart.py trained and serialized three sklearn Pipeline objects. Each pipeline contains preprocess and model steps. A repository search found no application import or load of these .pkl files, so they are separate training artifacts rather than the live prediction implementation.

Table 14: Saved model summary

Target	Task	Selected estimator	Test metrics	Artifact
total_sales	Regression	Ridge	R2 1.000000; RMSE 0.010614; MAE 0.007332	models/total_sales_model.pkl
profit	Regression	Ridge	R2 0.753999; RMSE 5.019354; MAE 3.190698	models/profit_model.pkl
stockout_flag	Classification	RandomForestClassifier	Accuracy 0.638286; weighted F1 0.655691; weighted precision 0.677070; weighted recall 0.638286	models/stockout_flag_model.pkl

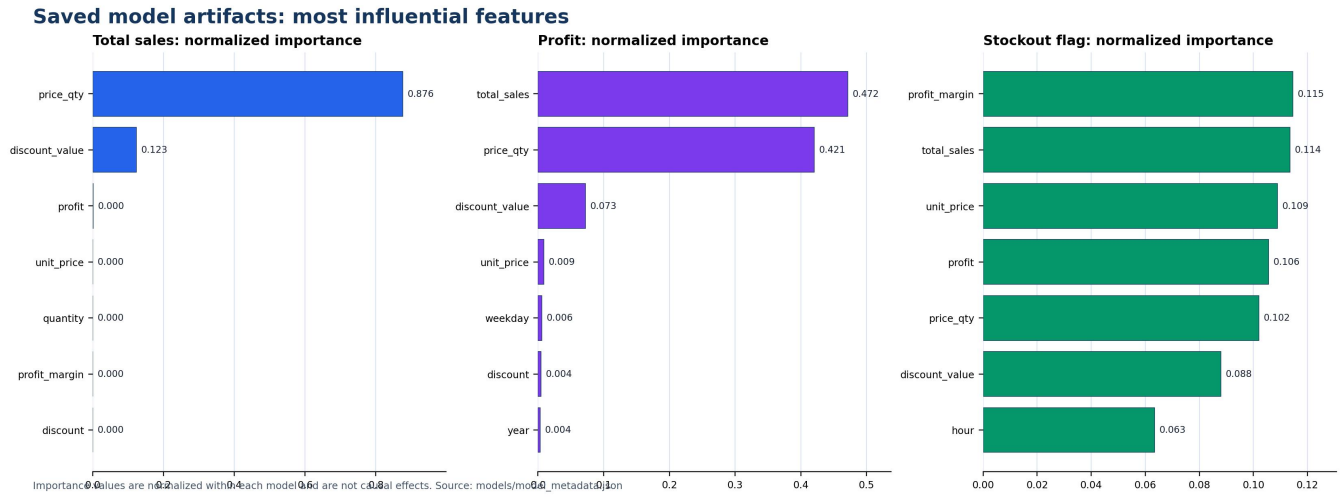

Figure 2: Top normalized feature importance recorded for each saved model

The feature-importance panels show that total_sales is dominated by price_qty (0.875938) and discount_value (0.123233). Those two engineered features reconstruct total_sales within about 0.005 due to rounding. Profit is dominated by total_sales (0.471716) and price_qty (0.420806). The stockout classifier spreads importance across profit_margin, total_sales, unit_price, profit, price_qty, discount_value, and time/category fields.

9.3 Saved model feature sets

total_sales

region, city, category, subcategory, unit_price, quantity, discount, profit, customer_type, payment_method, online_order, stockout_flag, weekday, month, year, hour, price_qty, discount_value, profit_margin

profit

region, city, category, subcategory, unit_price, quantity, discount, total_sales, customer_type, payment_method, online_order, stockout_flag, weekday, month, year, hour, price_qty, discount_value

stockout_flag

region, city, category, subcategory, unit_price, quantity, discount, total_sales, profit, customer_type, payment_method, online_order, weekday, month, year, hour, price_qty, discount_value, profit_margin

9.4 Stockout-classification behavior

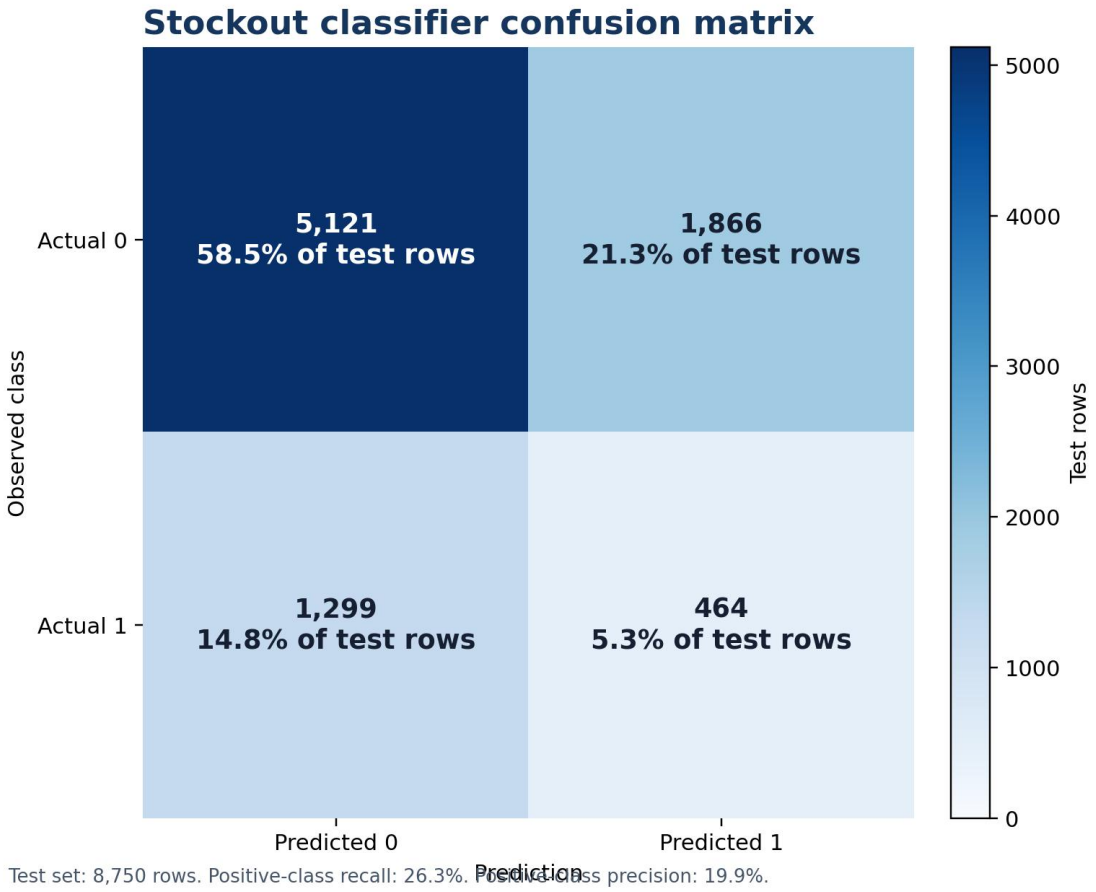

Figure 3: Stockout RandomForestClassifier confusion matrix

Although weighted metrics look moderate, the model identifies only 464 of 1,763 actual stockouts. Positive-class recall is 26.3% and positive-class precision is 19.9%. This model is not suitable for operational stockout decisions without better features, threshold tuning, calibration, and external validation.

Model validation conclusion

The saved artifacts demonstrate a working training pipeline, not production-ready forecasting. total_sales is leaked by engineered inputs, profit has moderate holdout fit, and stockout detection performs poorly on the minority class.

Evidence: scripts/train_retail_mart.py; models/model_metadata.json; models/.pkl; dataverse_backend/app/services/modeling.py; dataverse_backend/app/services/target_inference.py*

10. Explainable AI and decision tools

Table 15: Explainability and verification mechanisms

Capability	Method	Output	Limitation
Global feature importance	Normalized tree importance or absolute model coefficients	Ranked feature list	Association with model output, not causality
Local SHAP	SHAP TreeExplainer on up to 10 transformed test rows for tree-compatible models	Top signed contributors per sample	Not attempted for Ridge/Logistic models
Fallback XAI	Saved feature importance when SHAP is missing or fails	Top features and plain-language explanation	No local attribution
Counterfactual XAI	Deterministic single-feature search	Smallest found change that changes the prediction	Does not prove feasibility or causal effect
What-if	Apply percentage change to a numeric column and recompute business metrics	Baseline, scenario, delta, receipts	Scenario is mechanical, not a forecast
Root cause	Period comparison, dimension contribution ranking, price/volume/mix split	Drivers, chart, receipts, narrative	Descriptive decomposition, not causal identification
Data Quality Doctor	Rule-based duplicate/missing/constant/type checks	Issues, proposed fixes, before/after stats	Automated cleaning may change analytical meaning
Reproducibility certificate	SHA-256 fingerprints of data and selected results	Tamper-evident certificate and verification response	Proves repeatability under the same code/data, not real-world correctness
Provenance receipts	Formula, operation, source columns, row count, samples	Audit trail for KPIs and analytical facts	Sample rows are evidence, not a full lineage platform

Evidence: dataverse_backend/app/services/xai.py; counterfactual.py; whatif.py; root_cause.py; quality_doctor.py; certificate.py; provenance.py; audit.py

11. LLM and language-model integration

LLMs are optional. The application is designed so provider failure returns to deterministic templates rather than blocking calculations. Numeric facts are computed before narration and prompts instruct providers not to invent values.

Table 16: Configured language-model providers

Provider	Configured model/default	Use	Activation
OpenAI	gpt-4o-mini for chat and intent	Narration, intent refinement, titles, optional agent planning	OPENAI_API_KEY
Google Gemini	gemini-1.5-flash; report setting gemini-1.5-pro	Fallback narration and semantic/query refinement	GEMINI_API_KEY or GOOGLE_API_KEY
Anthropic	claude-sonnet-4-6 in config.py	Fallback narration	ANTHROPIC_API_KEY
DeepSeek	deepseek-chat	Intent parsing through OpenAI-compatible API	DEEPSEEK_API_KEY
DeepAnalyze	deepanalyze-8b; local fallback label phi3:mini	Remote/local OpenAI-compatible reasoning and narration	DeepAnalyze API or local base URL
Deterministic	Rule/template code	All calculations plus fallback summaries and answers	Always available

11.1 Provider order

In auto mode, llm_provider.py tries configured OpenAI, Gemini, Anthropic, and DeepAnalyze providers in that order. A specifically requested provider is placed first. Errors are recorded by provider type, and a missing/failed chain returns None so deterministic narration is used.

11.2 Agentic chat loop

When an LLM is configured, AgentLoop can choose deterministic tools, observe their computed JSON, and produce an answer restricted to those observations. The UI can display the thought/tool/argument/observation trace. Without an LLM, SessionService uses the deterministic answer path.

Configuration mismatch

config.py defaults CLAUDE_MODEL to claude-sonnet-4-6, while dataverse_backend/.env.example currently lists claude-3-5-sonnet-20241022. Deployment behavior follows the environment value when supplied, so documentation/configuration should be aligned.

Evidence: dataverse_backend/app/services/llm_provider.py; llm_service.py; agent_loop.py; query_planner.py; semantic_mapper.py; report_narrator.py; title_generator.py; app/core/config.py

12. Supabase, authentication, database, and storage

12.1 Authentication

Table 17: Authentication flow and security rules

Operation	Implementation
Signup	FastAPI validates input, then POSTs email/password/user metadata to Supabase Auth /auth/v1/signup
Password policy	At least 12 characters; uppercase, lowercase, digit, special character; cannot contain email local-part
Password storage	Supabase Auth owns hashing/storage; the application database does not store password fields
Email confirmation	Signup is rejected operationally if Supabase returns an immediate access token; confirmation link redirects to /login?confirmed=true
Resend	Supabase /auth/v1/resend with type=signup
Login	Supabase password grant returns access token, refresh token, expiry, and user
Refresh	Supabase refresh_token grant
Token verification	Legacy HS256 checked locally when JWT secret exists; asymmetric/new tokens checked against Supabase /auth/v1/user
Verification cache	Validated token-to-user mapping cached in memory for five minutes

12.2 PostgreSQL metadata schema

Table 18: Supabase application tables

Table	Important fields	Purpose
chat_sessions	id, user_id, title, status, active_dataset_id, metadata, timestamps	Ownership and conversation/workspace state
chat_messages	id, session_id, role, content, message_type, payload, created_at	User, assistant, system, and agent messages
datasets	id, session_id, user_id, filename, storage_path, file size, shape, columns, profile, semantic map, status	Dataset metadata and object location
agent_runs	id, session_id, dataset_id, agent_name, status, input, output, error, timestamps	Execution trace persistence
reports	id, session_id, dataset_id, title, type, format, storage_path, public_url, metadata	Generated report metadata
auth.users	Managed by Supabase	Credentials, confirmation state, and user identity

12.3 Object storage

- dataverse-datasets is a private bucket for uploaded dataset objects.
- dataverse-reports is a private bucket for generated HTML and PDF reports.
- The backend generates signed download URLs with a default one-hour lifetime.
- SUPABASE_SERVICE_ROLE_KEY remains backend-only and is used for REST and Storage operations.

- If Supabase is not configured, LocalPersistence stores table snapshots and files under session_storage/dataverse_chat.

Schema caveat

supabase/migrations/001_dataverse_schema.sql enables RLS but defines no browser policies because the backend service role bypasses RLS. dataverse_backend/supabase_schema.sql contains a similar schema with optional policies commented out. These duplicate schema sources should be consolidated before production migration management.

Evidence: supabase/migrations/001_dataverse_schema.sql; dataverse_backend/supabase_schema.sql; dataverse_backend/app/services/auth_service.py; supabase_client.py; session_service.py

13. Security assessment

Table 19: Implemented security controls

Control	Implementation	Assessment
Server-side password policy	12-character complexity and email-name exclusion	Strong baseline
Email verification	Required Supabase confirmation link	Prevents immediate unverified access when dashboard configuration is correct
No plaintext password storage	Credentials sent to Supabase Auth only	Correct responsibility boundary
JWT validation	HS256 local verification or Supabase user endpoint	Supports key-algorithm transition
IDOR guard	Session owner must match resolved identity	Protects owned session routes
Rate limiting	In-memory sliding window on sensitive POST routes	Useful for one process; not distributed
Security headers	Frame denial, MIME sniff prevention, referrer and permissions policy	Good browser baseline
Private buckets	Datasets and reports are non-public	Correct default
Service-role isolation	Service key is backend-only	Required for current persistence design
Upload checks	Size, extension, non-empty dataframe, domain guard	Reduces obvious misuse
Production error masking	Internal exception details hidden outside development	Reduces information leakage

Table 20: Security and production-hardening gaps

Priority	Gap	Risk	Recommended change
High	Tokens stored in localStorage	An XSS vulnerability could read access and refresh tokens	Prefer secure, HttpOnly, SameSite cookies or a hardened token strategy plus strict CSP
High	X-Dataverse-User accepted without bearer token	Caller-selected workspace identity can be spoofed	Require verified Supabase identity for all non-public data routes; remove anonymous identity fallback
High	Legacy sessions without owner remain open	Known session IDs may be accessible	Migrate/assign owners and deny ownerless sessions in production
High	Progress SSE route does not call _authorize	Session progress may be observable by ID	Verify bearer identity and session ownership before streaming
High	Service role bypasses RLS	Backend authorization errors expose	Keep strict application checks and

Priority	Gap	Risk	Recommended change
		all rows	add user-scoped defense-in-depth RPC/policies where feasible
Medium	In-memory rate limiter/cache	Resets on restart and is not shared across workers	Use Redis or another shared store in multi-instance deployment
Medium	Extension validation only	File content may not match extension	Add MIME/signature checks, workbook limits, decompression limits, and malware scanning
Medium	Two Supabase schema files differ	Migration drift	Choose one migration source and version every change
Medium	CORS includes HTTP localhost defaults	Suitable for development, not production	Use HTTPS production origins only
Medium	No explicit Content-Security-Policy	Reduced protection against script injection	Add a tested CSP in Next.js/FastAPI headers

14. Reporting, charts, and user-visible outputs

Table 21: Output types

Output	Produced by	Content
Chat answer	SessionService / AnalysisPipeline	Answer, summary, warnings, recommendations, next questions
KPI cards	business_metrics.py	Revenue, quantity, profit, margin, order value, counts, provenance
Tables	business/product/food/financial analysis	Rankings, frequencies, metrics, audit-friendly rows
Charts	data_quality.py and business modules	Bar, line, pie/donut, histogram, scatter, boxplot, feature importance, confusion matrix
Agent trace	agent_loop.py	Thought, tool, arguments, deterministic observation
Quality diagnosis	quality_doctor.py	Issue list, fixes, before/after
Certificate	certificate.py	Algorithm, data/result fingerprints, verified number count
HTML report	report_generator.py	Self-contained styled narrative, SVG/Chart.js-ready visuals, tables, evidence
PDF report	report_generator.py	ReportLab-rendered static report with fallback manual PDF writer

Chart specifications are normalized and validated before delivery. Invalid charts are dropped with warnings. ReportComposer tracks semantic fingerprints to prevent repeated charts, tables, or insights and adds data-specific explanations and takeaways.

*Evidence: dataverse_backend/app/services/data_quality.py; report_composer.py; report_generator.py; frontend/components/dashboard/**

15. Deployment and configuration

15.1 Local development

- npm run dev launches both the FastAPI backend and the Next.js frontend.
- FastAPI listens on http://127.0.0.1:8000 with reload enabled.
- Next.js listens on https://127.0.0.1:3000 using local certificate files.
- Next.js proxies /backend/* to the HTTP backend so browser requests remain same-origin HTTPS.

15.2 Containers

docker-compose.yml defines backend port 8000 and frontend port 3000. The backend image uses python:3.12-slim, installs requirements-mvp.txt, copies the application, and exposes an HTTP health check. The frontend image builds and starts Next.js. Current compose examples use HTTP frontend/backend URLs and should be fronted by a TLS reverse proxy in production.

15.3 Environment configuration

Table 22: Environment-variable categories

Category	Variables	Purpose
Application	ENVIRONMENT, APP_VERSION, ENABLE_OPENAPI_DOCS, REQUEST_TIMEOUT_SECONDS	Runtime behavior and API documentation
Transport	CORS_ORIGINS, CORS_ORIGIN_REGEX, TRUSTED_HOSTS, SECURE_HEADERS_ENABLED, HTTPS_REDIRECT	Browser/API transport controls
Rate limits	RATE_LIMIT_ENABLED, RATE_LIMIT_REQUESTS, RATE_LIMIT_WINDOW_SECONDS, RATE_LIMIT_PER_MINUTE	Abuse protection thresholds
Supabase	SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_JWT_SECRET	Authentication, PostgreSQL REST, Storage, JWT validation
Supabase storage	SUPABASE_DATASET_BUCKET, SUPABASE_REPORT_BUCKET, SUPABASE_HEALTH_TIMEOUT_SECONDS	Private bucket names and health checks
URLs	BACKEND_BASE_URL, FRONTEND_BASE_URL, NEXT_PUBLIC_DATAVERSE_API_URL, NEXT_PUBLIC_API_URL	API routing and confirmation redirects
Uploads/modeling	MAX_UPLOAD_SIZE_MB, RETAIL_ONLY_UPLOADS, AUTO_TRAIN_TARGET_CONFIDENCE, MIN_ROWS_FOR_PREDICTION	Dataset admission and modeling thresholds
LLM control	USE_LLM_NARRATION, LLM_PROVIDER, REPORT_NARRATOR_TIMEOUT_SECONDS, INTENT_LLM_PROVIDER, INTENT_LLM_TIMEOUT	Optional provider use
OpenAI	OPENAI_API_KEY, OPENAI_API_BASE, OPENAI_CHAT_MODEL, OPENAI_INTENT_MODEL	OpenAI narration/planning
Gemini	GEMINI_API_KEY, GEMINI_API_BASE, GEMINI_MODEL, GEMINI_REPORT_MODEL, GOOGLE_API_KEY	Google provider configuration
Anthropic	ANTHROPIC_API_KEY, CLAUDE_MODEL	Claude provider configuration
DeepSeek	DEEPSEEK_API_KEY, DEEPSEEK_API_BASE, DEEPSEEK_INTENT_MODEL	Intent parsing provider
DeepAnalyze	DEEPANALYZE_API_KEY, DEEPANALYZE_API_BASE, DEEPANALYZE_LOCAL_BASE_URL, DEEPANALYZE_MODEL	Remote/local OpenAI-compatible provider
Logging	LOG_DIR, LOG_LEVEL, LOG_JSON	Operational logs

Category	Variables	Purpose
Optional infrastructure	DATABASE_URL, REDIS_URL, CELERY_BROKER_URL, CELERY_RESULT_BACKEND, STORAGE_TYPE	Configured in settings but not part of the active session flow
Optional object storage	MINIO_*, AWS_*	Configured in settings but inactive in the current Supabase/local path
Observability	SENTRY_DSN, SENTRY_TRACES_SAMPLE_RATE, SENTRY_PROFILES_SAMPLE_RATE	Declared configuration; no active Sentry initialization found

Dependency reproducibility

The backend Dockerfile installs requirements-mvp.txt, whose packages are largely unpinned. requirements-full.txt contains many pins but is not the Docker input. Freeze the deployed requirements and remove duplicated/unneeded entries for reproducible production images.

*Evidence: frontend/scripts/dev.mjs; frontend/next.config.ts; docker-compose.yml; dataverse_backend/Dockerfile; frontend/Dockerfile; *.env.example*

16. Complete library and dependency inventory

The table records versions installed in the audited environment where available. Declared manifests are not fully consistent: several backend packages are unpinned in the active MVP requirements, and installed reportlab, joblib, and pytest-asyncio versions differ from requirements-full.txt.

Table 23: Important frontend and backend dependencies

Library	Audited version	Use status	Role
FastAPI	0.135.2	Direct	HTTP API, dependency injection, request validation, routing
Uvicorn	0.42.0	Runtime	ASGI development/production process
Pandas	2.3.3	Direct, 31 backend files	Dataframes, aggregation, profiling, CSV/Excel operations
NumPy	2.3.5	Direct	Numeric arrays, polyfit trends, model/XAI support
scikit-learn	1.8.0	Direct	Preprocessing, candidate models, metrics, train/test split
SHAP	0.51.0	Conditional direct	TreeExplainer local explanations
SciPy	1.16.3	Declared/supporting	Scientific stack dependency; not directly imported by active app files
OpenPyXL	3.1.5	Indirect	Pandas Excel reader for XLSX
ReportLab	4.5.1 installed	Direct	PDF generation and chart drawings
Jinja2	3.1.6	Direct	HTML report templating
HTTPX	0.28.1	Direct	Supabase Auth/REST/Storage, Gemini HTTP, DeepAnalyze, health checks
Pydantic	2.12.5	Direct	API and semantic/query schemas
pydantic-settings	2.13.1	Direct	Environment configuration
python-multipart	0.0.26	Framework support	Multipart file uploads
PyJWT	2.13.0	Direct	Legacy HS256 Supabase JWT validation
OpenAI	1.109.1	Optional direct	Report/query narration and agent

Library	Audited version	Use status	Role
			planning
google-generativeai	0.8.5	Optional direct	Gemini provider
Anthropic	0.100.0	Optional direct	Claude provider
Supabase Python	Not installed	Not required by current code	The application uses a custom HTTPX Supabase client instead
pytest	9.0.2	Test	Backend test runner
pytest-asyncio	1.4.0 installed	Test	Async endpoint/service tests
pypdf	6.14.2	Test/report validation	PDF inspection
Next.js	15.5.20 installed	Direct frontend	App Router, production build, local HTTPS dev server
React / React DOM	19.2.7 installed	Direct frontend	UI rendering and client state
TypeScript	5.9.3	Build	Static frontend types
Tailwind CSS	4.1.11	Direct frontend	Utility styling
Motion	12.42.2 installed	Direct frontend	Animations
Lucide React	0.553.0	Direct frontend	Icons
react-markdown	10.1.0	Direct frontend	Assistant/report markdown rendering
ESLint	9.39.1	Quality	Frontend static analysis

16.1 Declared but not part of the active core flow

Table 24: Supporting or inactive dependencies/configuration

Item	Repository status	Interpretation
SQLAlchemy / aiosqlite / DATABASE_URL	Declared in full requirements/settings; active session flow uses Supabase REST/local files	Not required for current UI flow
Redis / Celery	Settings exist; no active worker pipeline imported	Future/legacy configuration
MinIO / AWS S3	Settings exist; SessionService uses Supabase/local persistence	Inactive alternatives
Plotly / Kaleido / Matplotlib	Declared full stack; application reports use custom SVG/ReportLab; matplotlib appears through environment/tests	Not central to runtime chart delivery
Supabase Python package	Declared but not installed; no direct import	Custom HTTPX client is the actual implementation
requests	Declared; active external calls use HTTPX	Currently unnecessary for active app code
Sentry settings	Configuration fields exist; no Sentry initialization found	Not operational

17. Test and build verification

Table 25: Verification performed for this report

Command	Result	Evidence

Command	Result	Evidence
.venv/Scripts/python -m pytest -q	PASS: 214 tests; 14 dependency deprecation warnings; 140.50 seconds	Backend API, auth security, analytics, agents, reports, XAI, quality, root cause, what-if, uploads
npm run lint	PASS	ESLint completed without errors
npm run build	PASS	Next.js compiled, type-checked, and generated 10 static pages/routes
Dataset audit	PASS	35,000 x 21; zero missing cells; zero duplicate rows
Model artifact inspection	PASS	Three sklearn Pipeline objects with preprocess/model steps and expected estimators

The production build reports 102 kB shared first-load JavaScript. The dashboard route is approximately 102 kB route size and 208 kB first-load JavaScript, while public pages are about 109-110 kB first load.

Warnings

The 14 pytest warnings come from Matplotlib/PyParsing deprecations in installed dependencies, not failing application assertions. The Next build output says 'Skipping linting', but lint was run independently and passed.

18. Limitations, uncertainty, and robustness

Table 26: Technical limitations that affect interpretation

Area	Limitation	Effect
Dataset provenance	No original source description, license, collection period, sampling method, or encoding map	Business/generalization claims cannot be established
Saved total-sales model	Engineered features reconstruct the target	R2 = 1.0 is leakage, not forecasting evidence
Saved stockout model	Positive recall 26.3% and precision 19.9%	Misses most actual stockouts and generates many false alarms
Model validation	Single 75/25 random holdout; no cross-validation, temporal split, external set, calibration, or uncertainty intervals	Metrics may be unstable or optimistic
Live feature leakage	Safety check is mainly name/identifier based	Derived or hidden target proxies may still enter runtime models
Causal claims	Root cause and what-if are deterministic descriptive calculations	They do not establish causal effects
LLM behavior	Providers are optional and external output can vary	Narrative must remain bounded to computed facts
Persistence	Local fallback can hide missing Supabase configuration	A locally working app may not be production-ready
Security	Token storage and anonymous identity fallbacks remain	Production hardening is still required
Documentation	Historical docs describe routes/features that have changed	Code and current tests must remain the source of truth

19. Recommended next steps

1. Remove target-derived features from each saved training task, retrain, and use cross-validation plus a time-aware or external holdout where the business problem is temporal.
2. For stockout detection, optimize positive-class recall/precision, tune thresholds, report PR-AUC/ROC-AUC, calibrate probabilities, and validate on new stores/time periods.
3. Create a versioned dataset card containing origin, license, collection dates, feature definitions, categorical encoding maps, intended use, and prohibited use.
4. Require Supabase-authenticated identities for all dataset/session/report routes and remove X-Dataverse-User as an authorization mechanism.
5. Protect refresh/access tokens with HttpOnly secure cookies or an equivalent hardened browser session design, then add CSP and CSRF controls appropriate to that design.
6. Authorize the progress SSE route and deny access to ownerless legacy sessions.
7. Consolidate the two Supabase schemas into one ordered migration history and add defense-in-depth RLS policies where direct browser access is expected.
8. Pin requirements used by Docker, remove unused dependencies, and align .env.example defaults with config.py.
9. Add integration tests against a disposable Supabase project for confirmation email, JWT algorithms, RLS, private bucket access, and signed URLs.
10. Either integrate the saved retail artifacts as an explicit versioned inference feature or label/archive them as offline evaluation outputs to avoid architectural confusion.

20. Final implementation conclusion

The repository implements a substantial deterministic analytics application: real Supabase email/password authentication, a typed Next.js interface, a FastAPI session API, semantic dataset analysis, business KPIs, data-quality checks, runtime AutoML candidates, XAI, counterfactuals, root-cause decomposition, reproducibility receipts, and HTML/PDF reporting. The test and build results support functional completeness at the repository level.

The strongest engineering characteristic is separation of numerical computation from optional language-model narration. The main areas preventing a production-readiness claim are saved-model validation quality, incomplete dataset provenance, token/anonymous identity security, duplicate persistence schemas, and unpinned deployment dependencies.

Appendix A. Backend service-module inventory

Table 27: Backend service modules and status

Module	Status	Responsibility
agent.py	Legacy/alternate	Coordinates domain-specific dataframe tools for business analytics.
agent_loop.py	Conditional live path	LLM plan-act-observe loop that calls deterministic KPI, trend, quality, what-if, prediction, XAI, and root-cause tools.
ai_khata.py	Domain analytics	Detects and analyzes AI Khata-style transaction data.
ai_khata_tools.py	Domain tools	Question-answering helpers for AI Khata and transaction ledgers.
analysis_pipeline.py	Core active	Runs profiling, semantic mapping, planning, metrics, EDA, prediction, XAI, charts, narration, and response assembly.
analysis_planner.py	Core active	Chooses analysis depth and methods based on dataset evidence and query intent.
analytics_tools.py	Legacy support	Generic dataframe operations used by the older agent coordinator.
audit.py	Core active	Builds a flat audit trail from deterministic provenance receipts.
auth_service.py	Core active	Implements Supabase signup, email confirmation, login, refresh, JWT verification, and identity resolution.
business_leads_tools.py	Domain tools	Analytics helpers for business-leads datasets.
business_metrics.py	Core active	Calculates revenue, quantity, profit, margin, average order value, segment rankings, and query answers.
certificate.py	Core active	Creates SHA-256 dataset/result fingerprints and reproducibility certificates.
counterfactual.py	Core active after modeling	Searches for the smallest single-feature change that changes a prediction.
data_loader.py	Support	Loads CSV and Excel dataframes.
data_profiler.py	Core active	Profiles columns, missingness, uniqueness, distributions, preview rows, and semantic roles.
data_quality.py	Core active	Computes quality scores, EDA, IQR outliers, correlations, linear trends, and validated chart specifications.
dataset_classifier.py	Core active	Classifies semantic dataset type from roles and sample values.
deepanalyze_client.py	Optional	Calls a configured remote or local OpenAI-compatible DeepAnalyze endpoint.
domain_guard.py	Core active when enabled	Rejects datasets outside the retail/mart commerce domain.
health_checker.py	Operational	Checks dependencies and external service readiness.
intent_router.py	Core support	Routes natural-language questions to dataset-aware analysis intents.
llm_provider.py	Optional	Provider chain for OpenAI, Gemini, Anthropic, DeepAnalyze, then deterministic fallback.
llm_service.py	Optional/secondary	Simpler OpenAI/Gemini text-generation wrapper used by some modules.
modeling.py	Core conditional	Trains regression or classification candidates on an uploaded dataset and selects the best non-dummy model.

Module	Status	Responsibility
progress_bus.py	Core active	Publishes per-session Server-Sent Events for pipeline progress.
provenance.py	Core active	Builds formula, source-column, row-count, and sample-row receipts for computed values.
quality_doctor.py	Core active	Detects and optionally fixes duplicates, missing values, constants, and type problems.
query_planner.py	Core active	Converts a user question into metric, dimension, intent, filters, and report mode.
rate_limiter.py	Core active	In-memory sliding-window protection for authentication, upload, and message POST requests.
recommendation_engine.py	Core support	Generates evidence-grounded follow-up questions and recommendations.
report_composer.py	Core active	De-duplicates and organizes report sections, charts, tables, insights, and audit notes.
report_generator.py	Core active	Creates self-contained HTML and ReportLab PDF reports with tables and chart renderings.
report_narrator.py	Core active	Produces deterministic or provider-assisted summaries from computed facts.
response_builder.py	Support	Normalizes shared deterministic response payloads.
root_cause.py	Core active on request	Decomposes period changes by dimensions and price/volume/mix effects with receipts.
sales_tools.py	Domain tools	Sales-specific deterministic dataframe operations.
semantic_mapper.py	Core active	Maps arbitrary columns to roles and recognized dataset types, with optional LLM refinement.
session_service.py	Core orchestrator	Owns sessions, datasets, access checks, analysis, messages, reports, Supabase, and local fallback.
session_store.py	Core local runtime	Persists dataframe pickle/CSV files, metadata, and semantic maps per session and dataset.
supabase_client.py	Core active when configured	Custom backend REST and Storage client using the service-role credential, with local JSON/file fallback.
target_inference.py	Core conditional	Ranks safe regression/classification targets and rejects identifier-like candidates.
title_generator.py	Optional	Creates short session titles from dataset and query context.
whatif.py	Core active on request	Applies deterministic percentage scenarios to numeric columns and recomputes receipts.
xai.py	Core conditional	Uses SHAP TreeExplainer for tree models or normalized feature importance fallback, then counterfactual search.

Evidence: dataverse_backend/app/services/.py*

Appendix B. Model metadata detail

B.1 total_sales

Table 28: total_sales saved metrics

Metric	Value
r2	1.0
rmse	0.010614
mae	0.007332

Table 29: total_sales top recorded feature importance

Rank	Feature	Normalized importance
1	price_qty	0.875938
2	discount_value	0.123233
3	profit	0.000307
4	unit_price	0.000187
5	quantity	0.000148
6	profit_margin	0.000117
7	discount	0.000057
8	weekday	0.000002
9	customer_type	0.000002
10	year	0.000002

B.2 profit

Table 30: profit saved metrics

Metric	Value
r2	0.753999
rmse	5.019354
mae	3.190698

Table 31: profit top recorded feature importance

Rank	Feature	Normalized importance
1	total_sales	0.471716
2	price_qty	0.420806
3	discount_value	0.072588
4	unit_price	0.008904
5	weekday	0.005824

Rank	Feature	Normalized importance
6	discount	0.004445
7	year	0.003610
8	payment_method	0.003172
9	online_order	0.002641
10	quantity	0.001641

B.3 stockout_flag

Table 32: stockout_flag saved metrics

Metric	Value
accuracy	0.638286
f1_weighted	0.655691
precision_weighted	0.67707
recall_weighted	0.638286

Table 33: stockout_flag top recorded feature importance

Rank	Feature	Normalized importance
1	profit_margin	0.114652
2	total_sales	0.113537
3	unit_price	0.108866
4	profit	0.105698
5	price_qty	0.102105
6	discount_value	0.088025
7	hour	0.063389
8	month	0.050701
9	subcategory	0.038465
10	weekday	0.038168

Table 34: stockout_flag confusion-matrix cells

Actual	Predicted	Count
0	0	5,121
0	1	1,866
1	0	1,299
1	1	464

Appendix C. Evidence and source map

Table 35: Primary report evidence

Subject	Repository evidence
Architecture	docs/ARCHITECTURE.md; app/main.py; session_service.py; frontend/lib/dataverse-api.ts
Authentication	auth_service.py; auth_routes.py; frontend/lib/auth.tsx; tests/test_auth_security.py
Dataset ingestion	DatasetAgent; upload_parsing.py; data_loader.py; session_store.py
Analytics	analysis_pipeline.py; business_metrics.py; data_quality.py; data_profiler.py
Runtime models	modeling.py; target_inference.py; xai.py; counterfactual.py
Saved models	scripts/train_retail_mart.py; models/model_metadata.json; models/*.pkl
Retail data	data/retail_mart_processed_v1.csv
Supabase	supabase/migrations/001_dataverse_schema.sql; supabase_client.py; session_service.py
Reports	report_composer.py; report_generator.py; report_narrator.py
Frontend	frontend/app/*; frontend/components/*; frontend/lib/*; frontend/package*.json
Deployment	Dockerfiles; docker-compose.yml; scripts/dev.mjs; .env.example files
Verification	dataverse_backend/tests/*; npm lint/build output; model/dataset audit commands

End of report.